
DETERMINISMO EM SISTEMAS EMBARCADOS

Por que o estilo de código do Robótica Trator 2012 é raro hoje – e por que ainda importa

Autor : Marcos Roberto Braga
Projeto : Robótica Trator 2012 – vencedor de competição de robótica
Hardware : PIC16F84 (1 KB flash, 68 bytes RAM, 4 MHz)
Data : 2026
Licença : Material didático – livre para uso educacional

SUMÁRIO

1. Introdução
 2. O que é determinismo em sistemas embarcados
 3. Por que esse estilo é raro hoje
 4. Onde esse estilo é obrigatório
 5. A filosofia voltou – com outros nomes
 6. Duas culturas convivendo
 7. Por que ensinar isso hoje
 8. Onde esse estilo ainda vive (e paga bem)
 9. Conclusão
-

1. INTRODUÇÃO

Em 2012, um robô trator controlado por um PIC16F84 – microcontrolador de

8 bits com 1 KB de flash e 68 bytes de RAM – venceu uma competição de robótica.

O feito não chamaria atenção pelo hardware: o PIC16F84 é modesto até para os padrões da época. O que chama atenção é o MÉTODO empregado no software.

Este documento discute por que o estilo de programação usado nesse projeto é raro nos dias atuais, onde ele ainda é obrigatório, e por que vale a pena ensiná-lo a novos desenvolvedores.

2. O QUE É DETERMINISMO EM SISTEMAS EMBARCADOS

Determinismo, em sistemas de tempo real, significa que o comportamento temporal do sistema é:

- CONHECIDO A PRIORI – sabe-se exatamente quando cada tarefa vai rodar.
- REPRODUZÍVEL – o mesmo cenário produz o mesmo resultado.
- PREVISÍVEL – o pior caso (WCET) é mensurável e respeitado.

O estilo de código do Robótica Trator 2012 é determinístico porque:

- Usa escalonamento cíclico (cyclic executive) – cada tarefa tem período fixo, múltiplo de 1 ms.
- A ISR do Timer0 roda a cada 1 ms e é a base de tempo de TODO o sistema.
- Não há RTOS, não há preempção, não há mutex pesado.
- Não há alocação dinâmica de memória.
- Não há chamadas bloqueantes dentro de tarefas críticas.

Em resumo: nada acontece por acaso. Tudo é projetado.

3. POR QUE ESSE ESTILO É RARO HOJE

O estilo de código do Robótica Trator 2012 praticamente desapareceu do dia a dia da maioria dos desenvolvedores. Os motivos principais:

- a) Popularização do Arduino e do ESP32
 - `delay()`, `millis()` e RTOS prontos criaram uma geração que nunca precisou escrever uma ISR ou um escalonador.
- b) Frameworks modernos (Zephyr, FreeRTOS, Mbed)
 - Abstraem quase tudo. O programador raramente toca no hardware.
- c) Placas com recursos abundantes
 - Há tantos ciclos, tanta RAM e tanto flash que "desperdiçar" deixou de ser problema.
- d) Migração da indústria para ferramentas de TTA
 - Time-Triggered Architectures geram o escalonamento automaticamente. O engenheiro configura; a ferramenta gera o código.
- e) Cultura de "time-to-market"
 - A pressão por entregar rápido favorece frameworks prontos em detrimento de código sob medida.

O resultado: hoje é RARO ver um desenvolvedor capaz de escrever, do zero, um escalonador cíclico com ISR de 1 ms – como o do Robótica Trator 2012.

4. ONDE ESSE ESTILO É OBRIGATÓRIO

O estilo determinístico não é "coisa do passado". Ele é requisito legal e normativo em setores inteiros:

Domínio	Norma	Por que exige determinismo
Aviônica	DO-178C	Falha = morte. Certificação
Automotivo	ISO 26262	WCET precisa ser provado
Dispositivos médicos	IEC 62304	Marca-passo não pode ter jitter
Ferroviário	EN 50128	Sistemas fail-safe
Espacial	ECSS	Radiação + recursos limitados
Nuclear	IEC 61513	Redundância + previsibilidade

Nesses setores, "não pode errar" é literal. Um `delay()` mal colocado pode custar vidas. Um jitter de 2 ms pode derrubar um drone. Um mutex mal projetado pode travar um marca-passo.

O critério real, portanto, não é "sistema crítico". É:

PREVISIBILIDADE TEMPORAL OBRIGATÓRIA.

5. A FILOSOFIA VOLTOU – COM OUTROS NOMES

Muita gente acha que essa filosofia morreu. Não morreu. Mudou de nome e de ferramenta:

- RUST EMBARCADO (embedded-hal, RTIC)
 - Traz de volta o determinismo, agora com segurança de tipos e verificação em tempo de compilação.
- ZEPHYR RTOS com time-partitioning
 - Oferece determinismo com abstração moderna.
- AUTOSAR Classic
 - Escalonamento cíclico formalizado, usado em praticamente todo carro moderno. É o neto direto do estilo do Robótica Trator 2012.
- MISRA C
 - Conjunto de regras que força o programador a escrever código previsível – sem recursão, sem alocação dinâmica, sem goto.

O ESTILO é o mesmo. A FERRAMENTA mudou.

6. DUAS CULTURAS CONVIVENDO

A indústria de software hoje tem duas culturas lado a lado:

CULTURA "ABSTRATA"	CULTURA "DETERMINÍSTICA"
Web, mobile, cloud Prioriza time-to-market Aceita jitter Usa frameworks pesados	Embarcado crítico, tempo real Prioriza previsibilidade Combate jitter Usa escalonadores próprios

RTOS é a solução "Funciona, então está bom"	RTOS é parte do problema "Funciona E é provável, então ok"
--	---

O Robótica Trator 2012 está inteiramente na segunda coluna.

7. POR QUE ENSINAR ISSO HOJE

Ensinar esse estilo hoje é contra-cultural – e é justamente por isso que vale a pena. Porque:

1. Força o aluno a entender o que está por baixo das abstrações.
2. Ensina que recurso não substitui método.
3. Prepara para áreas onde a abstração é proibida (aviônica, automotivo, médico).
4. Desenvolve disciplina mental que se aplica mesmo em código não-crítico.

FRASE PARA OS ALUNOS:

"Você não precisa programar assim todos os dias. Mas precisa saber fazer isso quando o dia exigir."

8. ONDE ESSE ESTILO AINDA VIVE (E PAGA BEM)

Se algum aluno se interessar por esse estilo, os caminhos profissionais são:

- AEROESPACIAL – Embraer, Boeing, Airbus, INPE, SpaceX

- AUTOMOTIVO – Bosch, Continental, VW, GM, Stellantis
- MÉDICO – Philips, Siemens Healthineers, Medtronic
- INDUSTRIAL – Siemens, Rockwell, ABB, Schneider
- DEFESA – AEL, Avibras, IMBEL, Marinha, Aeronáutica

Nesses setores, esse estilo de código não é curiosidade histórica. É requisito de contratação.

9. CONCLUSÃO

O estilo de código do Robótica Trator 2012 é raro hoje – mas não por ser obsoleto. É raro porque a maioria dos desenvolvedores nunca precisou escrever código com exigência temporal forte.

Em sistemas críticos, esse estilo é obrigatório. Em sistemas modernos não-críticos, ele está renascendo com novos nomes (Rust embarcado, AUTOSAR, MISRA). E para quem ensina, ele é uma ferramenta poderosa para mostrar que:

DETERMINISMO NÃO SE COMPRA – SE PROJETA.

Um PIC16F84, um timer de 1 ms e método foram suficientes para vencer uma competição em 2012. Essa lição continua válida.

REFERÊNCIAS SUGERIDAS

- KOPETZ, Hermann. Real-Time Systems: Design Principles for Distributed

Embedded Applications. Springer, 2011.

- LIU, Jane W. S. Real-Time Systems. Prentice Hall, 2000.
- BARR, Michael. Programming Embedded Systems. O'Reilly, 2006.
- DO-178C – Software Considerations in Airborne Systems and Equipment Certification. RTCA, 2011.
- ISO 26262 – Road vehicles – Functional safety. ISO, 2018.

FIM DO DOCUMENTO
